

Label names can be used with goto statements anywhere in the function in which the label is declared. Labels are declared implicitly by writing `label_name:` followed by a statement. Label names must be unique within a function.

Function prototype: Identifiers declared within the list of parameter declarations in a function prototype (not part of a function definition) have function prototype scope. This scope ends at the end of the function prototype.

Visibility

The visibility of an identifier is that region of the program source code from which legal access can be made to the identifier's associated object. Scope and visibility usually coincide, though there are circumstances under which an object becomes temporarily hidden by the appearance of a duplicate identifier: the object still exists but the original identifier cannot be used to access it until the scope of the duplicate identifier is ended.

Technically, visibility cannot exceed scope, but scope can exceed visibility. Take a look at the following example:

```
void f (inti) {
    int j;          // auto by default
    j = 3;          // inti and j are in scope and visible

    {              // nested block
        double j;  // j is local name in the nested block
        j = 0.1;   // i and double j are visible;
                  // int j = 3 in scope but hidden
    }

    // double j out of scope
    j += 1;        // int j visible and = 4
}
// i and j are both out of scope
```

4.1.4 Name spaces

Name space is the scope within which an identifier must be unique. C uses four distinct categories of identifiers:

1. goto label names. These must be unique within the function in which they are declared.
2. Structure, union, and enumeration tags. These must be unique within